

Time and Space complexity

Order complexity Analysis -

Amount of space or Time taken up by an algorithm/
Code as function of input size.
NOT the actual time taken.

Case ① Unsorted array

2	3	1	7	4
---	---	---	---	---

 largest
linear - $n = 5$ operations $n = 5$

Time for linear search in worst case will remain
same even if our array was sorted.

Linear Search (largest)

$n \uparrow$ operation $\uparrow$

$n = 100$ time = 100 (operation)

$n = 10^5$ time = 10^5 (operation)

$O(n)$ *

Case-② Sorted array -

Constant operation.
 $n = 6 \rightarrow 1$ operation

arr =

1	2	3	4	5	6
---	---	---	---	---	---

 largest

largest = arr[n-1];

Here Input = 6 but Time/operation is
Constant.

$n = 5000$, Time/op = 1

$n = 10^5$ Operation = 1

Case ①

Time $\uparrow$ $n \uparrow$

$T \propto n$

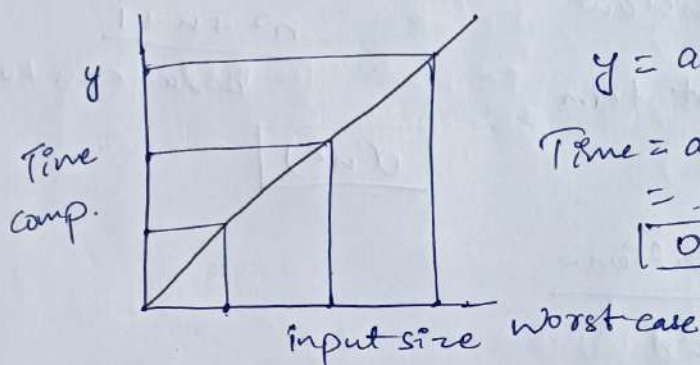
Case ②

$n \uparrow$ Time constant

T is constant.

Case ② is more efficient as compare to Case 1 because Case ② is not totally depend on input size. Whatever we take input but operation ~~will~~ will remain same. (sorted array)

For linear search -



$$y = ax + b$$

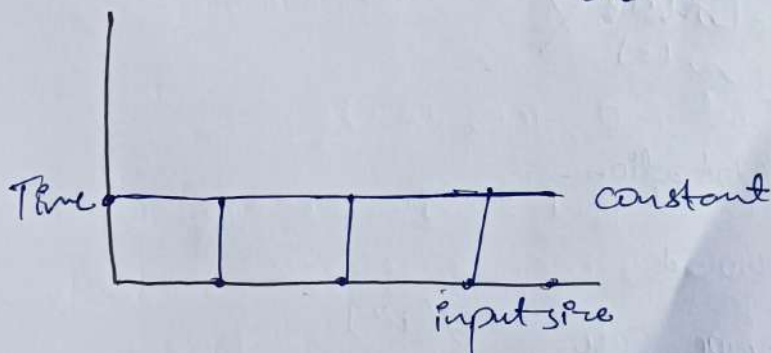
$$\text{Time} = ax + b$$

$$= n$$

$$\boxed{O(n)} \text{ unsorted array}$$

Sorted Array -

$$O(k) \rightarrow k \approx 1 = O(1)$$



Big O Notation :-

→ Always try to find worst case complexity.

Time complexity eqn -

$$f(n) = an^2 + bx + c \text{ or } y = ax + b,$$

To convert this function in Time C. we have to follow some steps which is given below -

- ① Ignore constant
- ② Take largest term

$$\frac{an^2 + bn + c}{n^2 + n + 1}$$

bad term ko lena

$$\boxed{O(n^2)}$$

Big Omega Notation -

Upper bound

best case

$$\Omega(n^2), \Omega(n^3), \Omega(n^4) \dots$$

$$\begin{matrix} \Omega(n) \\ \Omega(1) \end{matrix} \times$$

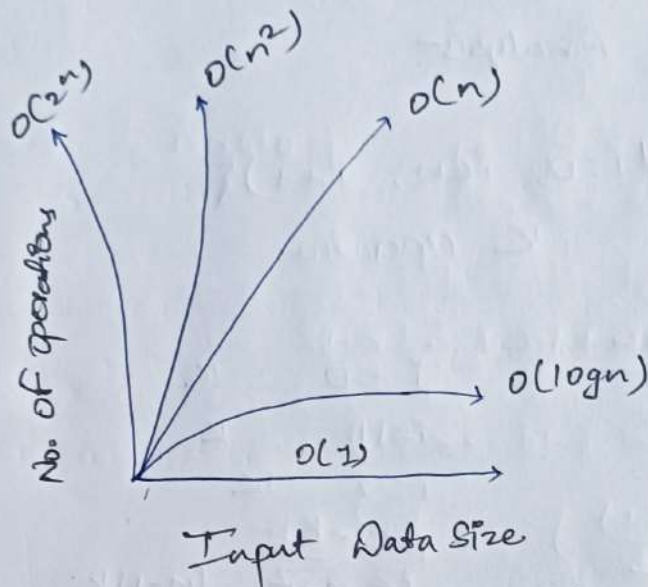
Big Theta Notation -

tight bound

average case

$$\Theta(n^2)$$

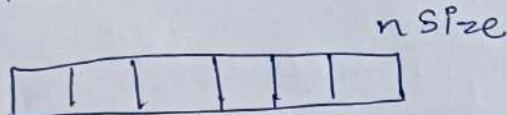
Common complexities :-



$O(1) < O(\log n) < O(n) < O(n^2) < O(2^n)$
Best Case TC. Worst Case TC.

Space Complexity :-

$\boxed{\text{input space}} + \boxed{\text{auxiliary space}} \text{ (Extra space)}$



Space Complexity $O(n)$ Expressed in Big-O-Not.

Space Complexity measures how much memory an algorithm needs as a function of input size (n), just like time complexity measures how much time it takes.

Theoretical Analysis

Simple loop Analysis-

```
for(int i=0; i<n; i++){  
    K operation  
}
```

$i=0$ K

$i=1$ K

$i=2$ K

$\vdots$

$i=n-1$ $(n-1)K$

$O((n-1)K)$

$TC = O(n)$

Nested loop Analysis-

① Outer loop (worst)

inner loop (worst)

② total operation runs

```
for(int i=0; i<n; i++){
```

```
    for(int j=i+1; j<n; j++){
```

Some constant work

```
    }  
}
```

$$i=0 \rightarrow n-1$$

$$i=1 \rightarrow n-2$$

$$i=2 \rightarrow n-3$$

$$i=n-1 \rightarrow 0$$

$$(n-1) + (n-2) + (n-3) \dots - 1 + 0$$

$$\text{Total} \rightarrow \frac{n(n+1)}{2}$$

$$\Rightarrow \frac{n^2 + n}{2}$$

$$O\left(\frac{n^2}{2} + \frac{n}{2}\right)$$

$$TC = O(n^2)$$

Nested Loop Analysis 2

```
for(int i=0; i<n; i++)
```

```
{
```

```
  for(int j=0; j<i; j++) {
```

```
    // some constant work
```

```
  }
```

Here, $i=0$ to $n-1$

$j=0$ to $i-1$

$i=0$ 0 times

$i=1$ 1x

$i=2$ 2x

$i=3$ 3x

$j=0$ to 0

$j=0$ to 1

$j=0$ to 2

K

$K=2K$

$K=3K$

$$\begin{array}{cccc}
 \vdots & & & \\
 i=5 & 5x & j=0 \text{ to } 4 & k=5k \\
 \vdots & \vdots & \vdots & \vdots \\
 i=n-1 & (n-1) & j=0 \text{ to } n-1 & k=(n-1)k
 \end{array}$$

$$k + 2k + 3k + 4k + \dots + (n-1)k$$

$$k(1 + 2 + 3 + 4 + \dots + n-1)$$

$$k = \left(\frac{n(n-1)}{2} \right) = \frac{kn^2}{2} - \frac{kn}{2}$$

Constant ignore

$$= n^2$$

$$T_c = O(n^2)$$

Time complexity of logarithmic-

Section - A

Complexity Analysis of Previous Modules -

① Linear Search:-

① Time Complexity

Best case $\rightarrow O(1)$

Worst case $\rightarrow O(n)$

② Space Complexity

$O(1)$

Explanation - Best case $O(1)$, Here array may be sorted and if I know array is sorted, so the optimize way is largest = arr[n-1], where we will get our largest element.

Worst case may be occur in that case where our searching element is not present in array or, searching is start from first index to last index.

② Binary search -

Time complexity

Space complexity - $O(1)$

Best case : $O(1)$

Worst case $\rightarrow O(\log n)$

Best case $\rightarrow$ Binary search is applicable for only sorted array.

element will get first, last or mid position.

Worst case $\rightarrow$ It will get the two half part of sorted array left and right.

Bubble sort:-

Time complexity-

Best case $\rightarrow O(n)$

Worst case $\rightarrow O(n^2)$

Space complexity-

$O(1)$

Worst case $\rightarrow$ Nested loop compares adjacent element and swaps.

This is continue at last pass which is $(n-1)$.

Selection sort:-

Time complexity-

Best case - $O(n^2)$

Worst case - $O(n^2)$

Both are same complexity, because the Nested loop compares from $i = \text{min pos.}$ to j last position.

In there compare minimum element and maximum will swap. again $i++$ and $j++$, this is will continue until last position.

Space complexity-

$O(1)$ in place

Insertion Sort :-

Time Complexity -

Best case $\rightarrow O(n)$ $\rightarrow$ When array will be sorted.

Worst case $O(n^2)$ $\rightarrow$

We should ~~let~~ first index is sorted and then compare from another.

Shifts element in back in sort part.

one outer loop $\times$ inner loop. $(n \times n) = O(n^2)$

Space Complexity $- O(1)$

Merge Sort :-

Time complexity

Best and worst $\rightarrow O(n \log n)$

Splits array ($\log n$ levels), merge $O(n)$ per level.

Space complexity -

need auxiliary array for merging
 $O(n)$

Quick sort:-

Time complexity-

Best case - $O(n \log n)$

Worst case - $O(n^2)$ (occurs when the pivot chosen is always the smallest or largest element eg. already sorted array with a naive pivot choice)

Space complexity - $O(n)$ (unbalance recursion depth)

Pattern Problem Complexity

- ① Square Numbers $O(n^2)$
- ② Incrementing Row $O(n^2)$
- ③ Right Triangle $O(n^2)$
- ④ Inverted Diamond $O(n^2)$
- ⑤ Sandwich Pattern $O(n^2)$
- ⑥ Incrementing Diamond $O(n^2)$

All pattern complexity is same because printing any 2D shape height ~~n~~ requires

visiting roughly ($n \times n$) nested loops.

Space complexity is $O(1)$.

Section - B

- ① Given a positive integer N , print all the numbers from 1 to N in increasing order, separated by a single space.

```
import java.util.*;

public class PrintNumbers {
    public static void main (String args[])
    {
        Scanner sc = new Scanner (System.in);
        int n = sc.nextInt();

        for (int i = 1; i <= n; i++) {
            System.out.print (i + " ");
        }
        System.out.println();
        sc.close();
    }
}
```

The numbers from 1 to 10 are printed in increasing order.

Time Complexity -

$O(n)$ → Loop is working constant.
Print i from 1 to n .

Space Complexity -

$O(1)$ constant space.

```
1 import java.util.*;
2
3
4 public class PrintNumbers {
5
6
7     public static void main (String args[]) {
8         Scanner sc = new Scanner(System.in);
9
10        int n = sc.nextInt();
11
12        for(int i = 1; i <= n; i++) {
13            System.out.print(i + " ");
14        }
15        System.out.println();
16
17        sc.close();
18
19
20
21
22
23
24 }
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Aashish\OneDrive\Desktop\java> cd "c:\Users\Aashish\OneDrive\
10
1 2 3 4 5 6 7 8 9 10
PS C:\Users\Aashish\OneDrive\Desktop\java>
```


Problem Statement:-

Given a positive Integer N , find the sum of the first N natural numbers using a loop

```
import java.util.*;

public class PrintNumbers {

    public static void main (String args[])
    {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int sum = 0;
        for (int i = 1; i <= n; i++) {
            sum += i;
        }
        System.out.print(sum + " ");
        System.out.println();
        sc.close();
    }
}
```

Explanation-

The Sum of the first five natural numbers is:
 $1 + 2 + 3 + 4 + 5 = 15$

Time complexity -

$O(n)$

Simple loop with a constant. print the sum.
From i to n .

Space complexity is same $O(1)$.

```
1  PrintNumbers.java  PrintNumbers  main(String[])
2  import java.util.*;
3
4  public class PrintNumbers {
5      Run | Debug
6      public static void main (String args[]) {
7          Scanner sc = new Scanner(System.in);
8
9          int n = sc.nextInt();
10         int sum = 0;
11
12         for (int i = 1; i <= n; i++) {
13
14             sum += i;
15         }
16         System.out.print(sum + " ");
17         System.out.println();
18
19         sc.close();
20
21     }
22 }
23
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Aashish\OneDrive\Desktop\java> cd "c:\Users\Aashish\OneDrive\Desktop\java"
PS C:\Users\Aashish\OneDrive\Desktop\java>
```

Problem Statement

Given two integers A and B, calculate the value of $A^{^B}$ (A raised to the power B using a loop).

```
import java.util.*;

public class PrintNumbers {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        int a = sc.nextInt();
        int b = sc.nextInt();

        int mul = 1;
        for (int i = 0; i < b; i++) {
            mul *= a;
        }
        System.out.print(mul + " ");
        System.out.println();
    }
}
```

Explanation-

The value of $2^{⁵}$.

Time Complexity-

$O(b)$ Because $i = 0$ to b . print mul. in a simple loop.

Space Complexity $\rightarrow O(1)$

```
1 import java.util.*;
2
3
4 public class PrintNumbers {
5     public static void main (String args[]) {
6         Scanner sc = new Scanner(System.in);
7
8         int a = sc.nextInt();
9         int b = sc.nextInt();
10
11         int mul = 1;
12
13         for(int i = 0; i < b; i++) {
14             mul *= a;
15         }
16         System.out.print(mul + " ");
17
18         System.out.println();
19
20         sc.close();
21     }
22 }
23
24
25
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Aashish\OneDrive\Desktop\java> cd "c:\Users\Aashish\OneDrive\Desktop\java"
2 5
32
PS C:\Users\Aashish\OneDrive\Desktop\java>
```